

Rapport individuel — Itération 1

Alexis Briend (abd7852) — Projet Long SHDL

13 avril 2026

Contexte

Responsable de la **structure générale** de l'application et de l'**interprétation du SHDL** (SCRUM-23).
Objectif sprint 1 : poser les fondations techniques (arborescence, gestion de dépendances, grammaire SHDL et parser vers un AST exploitable par les modules aval).

Activités réalisées

- **Squelette JavaFX/Gradle** (src/main/java/fr/n7/shdl/, packages model/view/controller, module-info, build.gradle).
- **Grammaire SHDL LL(1)** (parser/111/grammar/) : règles déclaratives, calcul FIRST/FOLLOW, vérificateur de conflits LL(1).
- **AST immuable** (parser/111/ast/) : hiérarchie Node → Expression/Statement, champs final, List.copyOf, non-null garanti, Visitor typé <R>.
- **Parseur à descente récursive** avec lookahead 2 sur les instances et messages d'erreur contextuels (code + position + attendu).
- **Documentation** : spec docs/specs/, plan docs/plans/, bilan docs/sprint1/.

Résultats

- **107 tests JUnit verts** sur 23 classes (grammaire, tokens, AST, visiteur, invariants runtime, cas négatifs, bout-en-bout ET/BasculeD/FsmSync/DecBCD).
- **39 fichiers Java** sous parser/111/. Une limite grammaticale connue (TERM_REST, wildcard FSM / STAR de règle voisine) est documentée et couverte par un test négatif, reportée au sprint 2.
- Revue critique interne : cinq défauts détectés (traversal incomplet du DefaultVisitor, champs redondants, invariants manquants), tous corrigés avant fin de sprint.

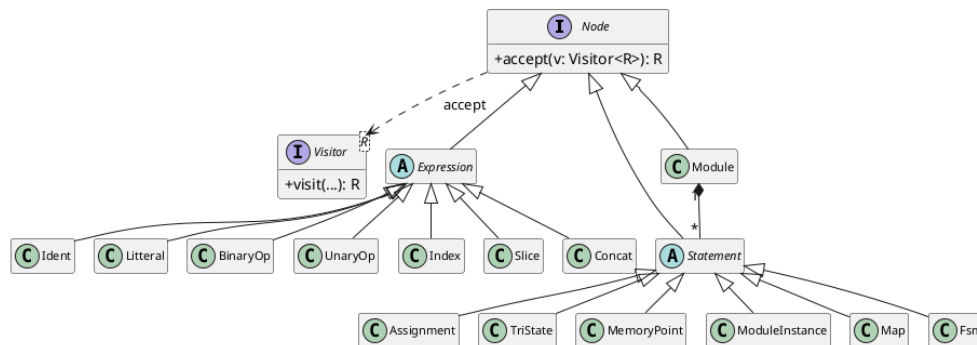


Figure — Diagramme de classes simplifié de l'AST (source ast-classes.puml).

Difficultés et réussites

Principale difficulté : garantir que la grammaire reste *réellement* LL(1) après chaque ajout de règle. Le vérificateur de conflits, écrit *avant* la grammaire, a détecté plusieurs ambiguïtés (instances de modules, FSM) sans aucune exécution du parser. Réussite : l'immuabilité stricte de l'AST, qui rend les passes aval (typage, automates) triviales à tester.

Manuel utilisateur

Pré-requis : JDK 21+, JUnit 4 et Hamcrest dans lib/. Compilation et tests via `javac/org.junit.runner.JUnitCore` (commandes détaillées dans docs/sprint1/). Usage : `new Parser(new Lexer(source)).parseFrom()` retourne l'AST racine `Module`, sur lequel un `Visitor<R>` exécute la passe aval choisie. Code sur `feature/skeleton-j`